GPS-Free Self-Healing Drone Networks via Virtual Spring-Dampers

Federico Corò, Riccardo Fabbian, and Claudio E. Palazzi
Department of Mathematics
University of Padua
Padua, Italy
federico.coro@unipd.it, fabb.riccardo@gmail.com, cpalazzi@math.unipd.it

***Abstract*—Distributed collective intelligence enables groups of autonomous agents to cooperatively achieve system-level goals through local interactions and decentralized decision-making. In this context, this paper presents a self-healing protocol for drone swarms that restores connectivity to a base station after link loss. The protocol combines ACK-based reachability detection, flooding-based hop discovery, periodic neighbor broadcasts, and a virtual spring-damper formation controller. Localization is GPS-free and uses a UWB PHY-layer channel with anchor beacons, Time-of-Flight ranging, and least-squares trilateration. We evaluate the system in *ns-3* on 15 single-base scenarios that vary topology, swarm density, ranging noise, and scheduled drone failures. The protocol restores connectivity within $\sim 1\,\mathrm{s}$ in every scenario; between the two attractive-force variants, healing latency is the same, while the weighted variant reduces mean return latency by 8% and its p_{95} tail by 21%.**

***Index Terms*—Distributed Collective Intelligence, Drone Swarm, UWB, Trilateration, Localization, Self-Healing.**

I. Introduction

Drone swarms represent a compelling instance of distributed collective intelligence, where autonomous agents cooperate through local sensing, communication, and decentralized decision-making to achieve system-level objectives. Compared to single drones, swarms provide larger coverage, redundancy, and modular task specialization, but they also require robust communication between drones and base stations, as well as coordinated formation control [1]. In decentralized swarm operations, communication links between individual drones and the base station may be disrupted by jamming, interference, or geographic occlusions. In such situations, the swarm must autonomously detect connectivity loss and collectively reorganize through a self-healing process that restores communication via a multi-hop relay chain, preventing network partitioning without centralized control.

To address this challenge, in this work-in-progress (WIP) paper we present a self-healing protocol in which drones detect the loss of base connectivity and cooperatively reconfigure through a virtual spring-damper controller, with localization performed without GPS via UWB Time-of-Flight trilateration against fixed anchors. The building blocks used here—UWB ranging, hop-discovery flooding, and spring-damper control—are individually well known; the contribution of this work is their integration into a single onboard loop in which GPS-free localization, ACK-based loss detection, and formation reconfiguration co-design the same self-healing decision, in contrast to prior work that addresses these aspects in isolation or relies on GPS / centralized coordination. We evaluate two attractive-force variants (centroid and weighted) in *ns-3* on a single-base batch of 15 scenarios that varies topology, swarm density, ranging noise, and scheduled drone failures.

The rest of this paper is organized as follows. Section II reviews the related literature; Section III presents the proposed self-healing solution; Section IV describes the experimental setup and Section V discusses the outcome of our tests; Section VI concludes and outlines next steps for this WIP.

II. Related Work

Restoring connectivity in mobile multi-agent systems has been studied across several research lines. In wireless sensor and actor networks, topology management surveys [2] and cascaded-relocation schemes [3] recover from node failures by triggering only the minimum number of nodes after a partition. Restore connectivity through mobile relays [4] has shown that deploying mobile actors to bridge disjoint segments is more efficient than complete redeployment, an insight that motivates self-healing of the relay-chain in aerial settings.

For unmanned aerial vehicles, formation control is based on classical flocking and potential-field methods. Olfati-Saber's flocking framework [5] and Khatib's artificial potential fields [6] provide the analytical basis that virtual spring-damper models [7], [8] extend to networked agents. Real flying swarms have demonstrated decentralized coordination using only local communication [9], [10], while Coppola *et al.* [11] showed that relative localization based on on-board communication is sufficient for safe MAV teaming.

GPS-free self-localization is a parallel concern. UWB ranging combined with anchor beacons supports centimeter-level positioning for aerial robots [12], [13]. Unlike topology-management schemes [2], [3] that assume GPS or centralized state, formation controllers [5], [7] that decouple control from network state, and UWB stacks [12], [13] evaluated on static topologies, our protocol couples loss-of-link detection, GPS-free localization, and reactive formation control in a single decentralized loop, autonomously restoring single-base connectivity after drone failures.

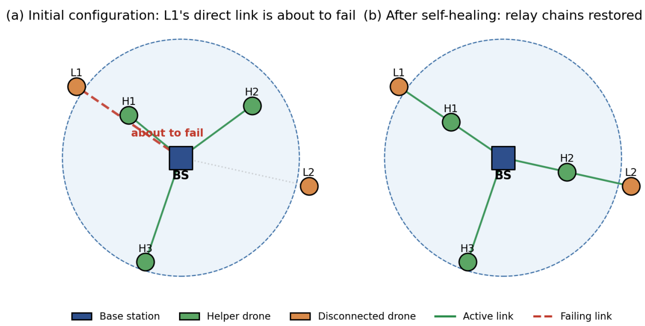

Fig. 1. Self-healing scenario. (a) Initial layout: helper drones (H1–H3) are inside coverage; L1 holds a direct link to the base that is about to fail (red dashed), while L2 is already isolated. (b) After the spring-damper repositioning, helpers move outward to bridge each disconnected drone with the base via a multi-hop relay.

III. Self-Healing Protocol

The protocol enables a drone to detect loss of communication with the base station and autonomously reorganize to form a multi-hop relay chain. It has two components: a loss-of-link detector and a formation controller based on a virtual spring-damper model. Upon ACK timeout, a drone runs the controller using neighbor positions and hop counts (obtained by flooding). The controller produces attractive forces toward the preceding and following hops in the relay chain, plus repulsive forces against close neighbors. The equilibrium is the geometric midpoint between the previous and next hop, which maximizes the Signal-to-Noise Ratio on both incident links. Figure 1 sketches a representative scenario: an in-coverage drone whose direct link is about to fail, and the post-healing relay chain that helper drones form to restore connectivity.

A. System Architecture

The architecture has three node roles: a stationary Base Station (B), static UWB Anchors (A_j), and mobile Drones (D_i) running an identical control stack. The base station also acts as an anchor. Following the Crazyflie 2.1 model [14], base traffic is unicast while swarm coordination uses broadcast. Messages are delivered over a UWB PHY-layer channel that adds propagation delay (d/c) and enforces a range cutoff. Each drone dispatches packets to four logical modules: hop-discovery flooding, a neighbor table keyed by neighbor ID, a UWB ranging/trilateration module, and core control logic for position updates, ACKs, and help-proxy requests. The controller reads UWB position and neighbor data and emits a velocity command through an actuator. The base station tracks position updates, replies with ACKs (carrying base position and hop 0), and periodically seeds floods.

B. Help-Proxy Triggering

Each drone sends periodic unicast `PositionUpdate` messages to the base and waits for an ACK. If no ACK arrives within a safety window of $1.5s$, the drone considers the link broken and broadcasts a "help-proxy" signal (`HELP_PROXY CORE` packet); any peer receiving it switches into mission mode. Broadcast matches the Crazyflie communication model.

After receiving the first relayed ACK addressed to itself, the requester is return-armed and waits a hop-staggered timeout proportional to $(H_{max} - h)$ (akin to 802.11p), where H_{max} is the maximum hop distance back to the base and h is the drone's own hop count, so the farthest drones return first and propagate a `RETURNING` flag that collapses the chain from the tail inward. During return, the attractive gain is reduced; on a direct ACK the drone enters station-keeping near the coverage boundary, re-arming if coverage is lost again to prevent oscillation.

C. Flooding-Based Hop Discovery

Flooding allows drones to estimate the hop distance to the base station even when direct connectivity is lost. The base periodically broadcasts a `START`; every in-coverage drone becomes an initiator and rebroadcasts a `DISCOVERY` with hop 1. Broadcast initiation (instead of unicast to a chosen seed) avoids relying on a single node that might be out of range. On `DISCOVERY` reception, a drone with a live direct ACK declares hop 1; otherwise it increments the carried hop. A node that detects an ACK timeout suppresses its stale "hop 1" until the next flood, preventing false attraction toward a dead link. Each node only reacts when a shorter path to the base arrives arrives for the same flood instance, then emits a `REPORT` and rebroadcasts `DISCOVERY`; reports are forwarded at most once per improvement, keeping the protocol lightweight.

D. Neighbor Information Exchange

For local, decentralized reactivity, each drone maintains a neighbor table fed by periodic `NEIGHBOR` broadcasts. The payload carries the sender ID, its hop count to the base, and UWB-trilaterated coordinates. When a drone has a direct ACK from the base, it injects a synthetic neighbor entry for the base (position from the ACK, hop 0), so the controller can treat the base as just another neighbor without special-casing.

E. Formation Control via Virtual Spring-Damper Model

Healing behavior is realized by a virtual spring-damper controller [7], [8] that combines linear attraction to neighbors at adjacent hop levels (the previous and next hops in the relay chain) with $1/d^2$ short-range repulsion against any neighbor closer than the safety distance D_{safe}:

$$\vec{F}_{tot} = \sum_{j \in N_{prev} \cup N_{next}} K_{att}(\vec{p}_j - \vec{p}_i) - \sum_{\substack{j \in N \\ d_{ij} < D_{safe}}} \frac{K_{rep}}{d_{ij}^2} \cdot \frac{\vec{p}_j - \vec{p}_i}{d_{ij}} \quad (1)$$

where $\vec{p}_i$ and $\vec{p}_j$ are the positions of drone i and neighbor j; N_{prev} and N_{next} are the sets of neighbors at the previous and next hop levels (hop $h-1$ and $h+1$); N is the set of all neighbors; $d_{ij} = \|\vec{p}_i - \vec{p}_j\|$ is the Euclidean distance between drones; K_{att} and K_{rep} are the attractive and repulsive gains (Table I); and D_{safe} is the minimum safe separation distance. The attractive term drives a drone toward the midpoint of its previous and next hops; repulsion is applied to all neighbors,

TABLE I
BASELINE SIMULATION PARAMETER VALUES

Parameter	Value
Coverage radius R_{max}	50 m
Coverage area	Circular, centered at base station
Number of drones N_d	20
Number of anchors N_a	18
Controller algorithm	centroid, weighted
Attractive gain K_{att}	0.3
Repulsive gain K_{rep}	8.0
Safety distance D_{safe}	2.0 m
Max speed V_{max}	1.0 m/s
Drone mass m	0.029 kg
Tick interval T_{ctrl}	0.05 s
Simulation duration T_{sim}	300 s
UWB noise σ_{uwb}	0.0 m

including peers with the same hop count. We compare two attractive variants: (i) *centroid* groups neighbors by hop count and applies one pull per group centroid, while (ii) *weighted* scales each pull by the opposite-side population so the midpoint stays the equilibrium even under unbalanced sides. The net force is converted to acceleration via Newton's second law and applied through a velocity actuator capped at V_{max}. A mission flag gates the controller: drones broadcast neighbor state while idle and switch on the spring-damper loop when a help-proxy trigger is received.

IV. Experimental Setup

The protocol was evaluated in a custom *ns-3* [15] scenario—chosen for its mature, time-accurate wireless PHY models suited to UWB Time-of-Flight evaluation—built around a direct UWB PHY-layer channel that delivers each transmission after a *d/c* propagation delay with a strict range cutoff. Static UWB anchors and the base station itself periodically broadcast position-tagged beacons; drones compute Time-of-Flight ranges and estimate their pose by least-squares trilateration. The baseline scenario uses one base at $(0, 0, 0)$, 20 drones, and 18 standalone anchors. Six drones act as helpers on a $\sim$40 m ring just inside the 50 m coverage radius; 11 drones start in the 68–82 m belt just outside coverage; three far scouts at 112–118 m force a multi-hop chain. Anchor placement guarantees at least three line-of-sight beacons everywhere. Each drone ticks at 0.05 s, performing a position update to the base, an ACK-timeout check (with `HELP_PROXY` on miss), and a controller step. The ACK timeout is 1.5 s, position updates are sent every 0.5 s or on movement, and anchors beacon at 2 Hz. A JSON declaration file runs batches that override positions and schedule mid-simulation drone failures. Table I lists the baseline parameters.

A. Mobility Model

To focus on network-driven coordination, drones use a lightweight kinematic mobility model that integrates commanded accelerations under standard Newtonian motion:

$$\mathbf{p}_{new} = \mathbf{p}_{old} + \mathbf{v}_{old}\Delta t + \tfrac{1}{2}\mathbf{a}\Delta t^2, \quad \mathbf{v}_{new} = \mathbf{v}_{old} + \mathbf{a}\Delta t \quad (2)$$

where $\mathbf{p}$ and $\mathbf{v}$ are position and velocity, $\mathbf{a}$ is the acceleration computed from Eq. (2), $\Delta t = T_{ctrl} = 0.05$ s is the control tick interval, and subscripts *old* and *new* are consecutive time steps.

V. Results and Analysis

We ran a single-base batch of 15 scenarios that covers baseline, dense, linear, tight, wide, and asymmetrically skewed swarm topologies; sparse anchor configurations; noisy ranging; heavy/slow drones; and scheduled drone failures (helper, relay, cascade, simultaneous-cluster kills, and a kill timed during the healing window). Each scenario was run exactly twice with deterministic configuration: once with the centroid controller and once with the weighted controller, for a total of 30 runs. All runs completed successfully. The simulator reports trilateration error, healing latency (time from `HELP_PROXY` to the first relayed ACK), return latency, recovery rate, final hop coverage, and station-keeping drift. Table II summarizes the batch.

TABLE II
BATCH METRICS ACROSS THE 15 SINGLE-BASE SCENARIOS (MEAN $\pm$ STD. DEV.).

Metric	Centroid	Weighted
Trilateration avg err. (m)	0.4708 ± 0.2526	0.4308 ± 0.2261
Healing latency avg (s)	0.9227 ± 0.1059	0.9247 ± 0.0981
Return latency avg (s)	23.2436 ± 12.04	21.36 ± 6.19
Recovery rate (%)	85.24 ± 28.11	83.84 ± 33.24
Final coverage (%)	98.63 ± 3.607	98.63 ± 3.607

The 85% mean recovery rate is bounded by three failure modes observed in the batch: (i) sparse partitions where no helper is within UWB range of the lost drone (*tight* and *wide* topologies), (ii) ranging-noise spikes that destabilize trilateration during the bridging window (*noisy_uwb_long*), and (iii) cascade kills that remove a freshly placed relay before its first relayed ACK propagates.

The protocol restores connectivity in roughly one second in every batch. Figure 2 shows that the two variants of the controller are tied in this metric: every scenario except *noisy_uwb_long* (007) sits on the hop-staggered wait floor of 0.95 s for both algorithms, and (007) itself differs by less than 0.04 s. The wait dominates the bridging time within a single base's coverage radius, so the centroid/weighted difference does not show up here.

The two variants do diverge on the *return* phase, where the lost drones move back into direct coverage. The weighted controller cuts mean return latency by 8% (21.4 s vs 23.2 s) and the p_{95} tail by 21% (27.6 s vs 35.1 s); its return-path efficiency is also tighter (1.05 ± 0.07 vs 1.12 ± 0.28). The recovery rate and the final coverage are similar. A simple reason is that the weighted scaling dampens the gradient as the chain collapses inward, reducing the late overshoot. The comparison under partitioned topologies is deferred to the multi-base extension (Section VI).

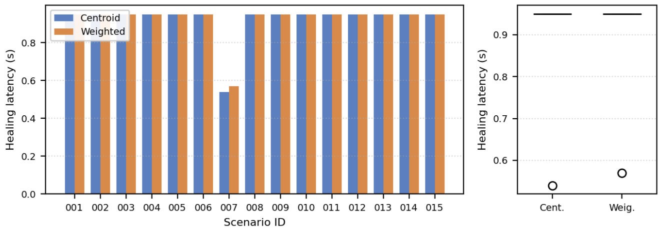

Fig. 2. Healing-latency distribution across the 15 scenarios. Left: per-scenario values for both controllers. Right: aggregate boxplot.

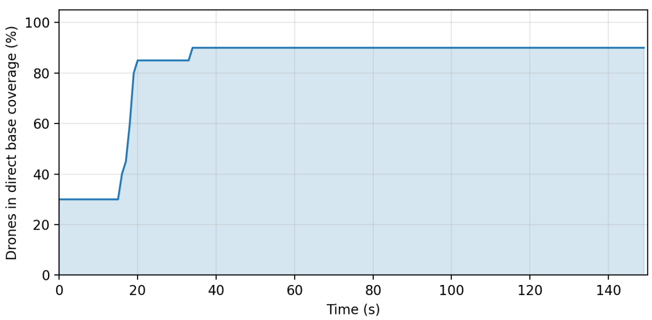

Fig. 3. Node connections over time in a representative run.

A. Parameter Sensitivity

The controller gains in Table I were selected by a grid search scoring oscillation amplitude, time-to-target, and minimum inter-drone distance. Higher K_{att} accelerates convergence but causes relay drones to overshoot the midpoint when V_{max} saturates; lowering K_{att} from 1.0 to 0.3 removed visible oscillations at $\sim$1 s extra settling. Conversely, large K_{rep} or D_{safe} prevent helpers from compressing into a relay chain near the coverage boundary. Drone mass (m) acts as damping through $\mathbf{a} = \mathbf{F}_{tot}/m$. A complete sweep is deferred to future work.

Figure 3 traces active connections during a representative run, showing the transition from isolated links to a restored multi-hop relay.

VI. Conclusions and Future Work

We presented a decentralized self-healing protocol that integrates ACK-based loss detection, flooding-based hop discovery, neighbor broadcasts, GPS-free UWB localization, and a virtual spring-damper controller. Across 15 single-base scenarios the protocol restores connectivity in $\sim$ 1 s; the weighted variant matches the centroid on healing latency but cuts mean return latency by 8%. Limitations include the need for at least one in-range peer per lost drone, dependence on anchor density, and a kinematic mobility model that ignores aerodynamics and multipath.

Several extensions are planned: *multi-base topologies* with per-base hop counts so each drone optimizes against its nearest reachable base; *base-station failure tolerance* via scheduled base kills and cascading-failure scenarios; *channel realism* replacing the range-cutoff UWB model with a path-loss/fading propagation model; a *full sensitivity sweep* over $(K_{att}, K_{rep}, D_{safe})$ trading healing latency against station-keeping drift; *network scaling* beyond 20 drones with heterogeneous roles; and *hardware validation* on Crazyflie 2.1 with Loco UWB anchors to quantify the sim-to-real gap.

References

[1] Q. Ouyang, Z. Wu, Y. Cong, and Z. Wang, "Formation control of unmanned aerial vehicle swarms: A comprehensive review," *Asian Journal of Control*, vol. 25, no. 1, pp. 570–593, 2023.

[2] M. Younis, I. F. Senturk, K. Akkaya, S. Lee, and F. Senel, "Topology management techniques for tolerating node failures in wireless sensor networks: A survey," *Computer Networks*, vol. 58, pp. 254–283, 2014.

[3] A. A. Abbasi, M. F. Younis, and U. A. Baroudi, "Recovering from a node failure in wireless sensor-actor networks with minimum topology changes," in *2011 IEEE International Conference on Communications (ICC)*. IEEE, 2011, pp. 1–6.

[4] F. Senel and M. Younis, "Optimized connectivity restoration in a partitioned wireless sensor network," in *2011 IEEE Global Telecommunications Conference (GLOBECOM)*. IEEE, 2011, pp. 1–5.

[5] R. Olfati-Saber, "Flocking for multi-agent dynamic systems: Algorithms and theory," *IEEE Transactions on Automatic Control*, vol. 51, no. 3, pp. 401–420, 2006.

[6] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," *The International Journal of Robotics Research*, vol. 5, no. 1, pp. 90–98, 1986.

[7] K. Piemngam, I. Nilkhamhang, and P. Bunnun, "A virtual spring damper method for formation control of the multi omni-directional robots in cooperative transportation," in *2019 11th International Conference on Information Technology and Electrical Engineering (ICITEE)*. IEEE, October 2019, pp. 1–6.

[8] Q. Chen, S. M. Veres, Y. Wang, and Y. Meng, "Virtual spring-damper mesh-based formation control for spacecraft swarms in potential fields," *Journal of Guidance, Control, and Dynamics*, vol. 38, no. 3, pp. 539–546, 2015.

[9] S. Hauert, S. Leven, M. Varga, F. Ruini, A. Cangelosi, J.-C. Zufferey, and D. Floreano, "Reynolds flocking in reality with fixed-wing robots: Communication range vs. maximum turning rate," in *2011 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2011, pp. 5015–5020.

[10] G. Vásárhelyi, C. Virágh, G. Somorjai, T. Nepusz, A. E. Eiben, and T. Vicsek, "Optimized flocking of autonomous drones in confined environments," *Science Robotics*, vol. 3, no. 20, p. eaat3536, 2018.

[11] M. Coppola, K. N. McGuire, C. De Wagter, and G. C. H. E. de Croon, "On-board communication-based relative localization for collision avoidance in micro air vehicle teams," *Autonomous Robots*, vol. 42, no. 8, pp. 1787–1805, 2018.

[12] M. Hamer and R. D'Andrea, "Self-calibrating ultra-wideband network supporting multi-robot localization," *IEEE Access*, vol. 6, pp. 22 292–22 304, 2018.

[13] V. Mai, M. Kamel, M. Krebs, A. Schaffner, D. Meier, L. Paull, and R. Siegwart, "Local positioning system using uwb range measurements for an unmanned blimp," *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 2971–2978, 2018.

[14] Bitcraze AB, "Crazyflie 2.x nano quadcopter," https://www.bitcraze.io, 2026.

[15] ns-3 Consortium, "ns-3 network simulator," https://www.nsnam.org, 2026.